

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

The minimum cost of reaching (m, n) from $(0, 0)$ is

$$\frac{m(m+1)}{2} + n.$$

The cost only depends on the x being left, so take all n north steps while still at $x = 0$, where each costs 1 and they total n , then the m east steps leave from $x = 0, 1, \dots, m-1$ in turn and cost $1, 2, \dots, m$; writing that series forwards and backwards pairs it into m columns of $m+1$, so it sums to $m(m+1)/2$. One path achieving the minimum is to take all n steps north from the origin first and then all m steps east, and it is the only one, since any north step taken after moving east costs more than 1 because it leaves from $x > 0$, and a detour into negative x adds at least one step west and one step back east, each costing at least 1, without making any other step cheaper.

1.b

1. False. The state space is infinite, but UCS processes states in order of past cost and every step costs at least 1, so a state with past cost below the goal's cost is within that many steps of the origin. There are only finitely many such states, so UCS reaches the goal and terminates.
2. False. UCS returns the minimum-cost path, but it explores outside the rectangle: $(-1, 0)$ and $(0, -1)$ each cost 1 to reach, which is cheaper than any goal beyond the origin, so they are processed even though they lie outside $0 \leq x \leq m, 0 \leq y \leq n$.
3. False. UCS does return the minimum-cost path, but it can explore locations whose past cost *equals* the minimum rather than being strictly less than it. When several states tie for the lowest cost, the queue's tie-break decides the order. For the goal $(1, 0)$, which costs 1, the states $(-1, 0)$, $(0, -1)$ and $(0, 1)$ also have past cost 1 and are popped before the goal.

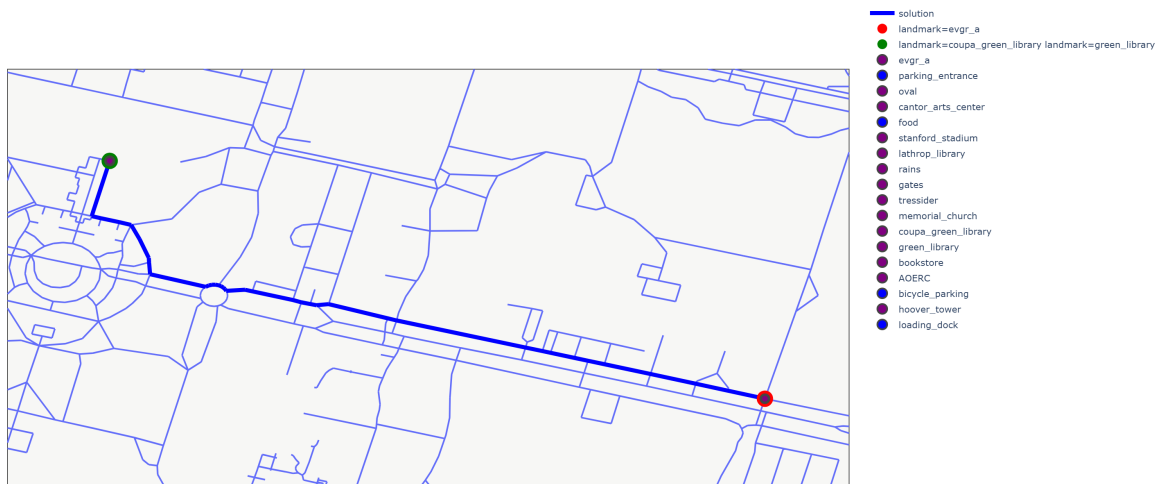
1.c

1. True. Adding a connection only adds paths and never removes one, so the minimum over the larger set of paths cannot exceed the minimum over the smaller set. The distance either drops, if the best path through the new connection is cheaper, or stays the same.
2. False. An action out of a state that the start cannot reach lies on no path from the start to the goal, however small its cost. For example, with $A \xrightarrow{5} C$ as the only route from start A to goal C and a separate component $D \xrightarrow{-1000} E$, the cheap action never appears in the minimum-cost path. The same holds for an action leading into a dead end.
3. False. Adding 1 to every action raises a path's cost by its number of edges, so paths with more edges are penalised more and the ranking can change. Take route 1, $A \xrightarrow{10} Z$ with one edge, and route 2, $A \xrightarrow{2} B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{2} Z$ with four. Before the change route 1 costs 10 and route 2 costs 8, so route 2 is the minimum. After adding 1 to every action, route 1 costs 11 and route 2 costs 12, so route 1 becomes the minimum-cost path.

2.b

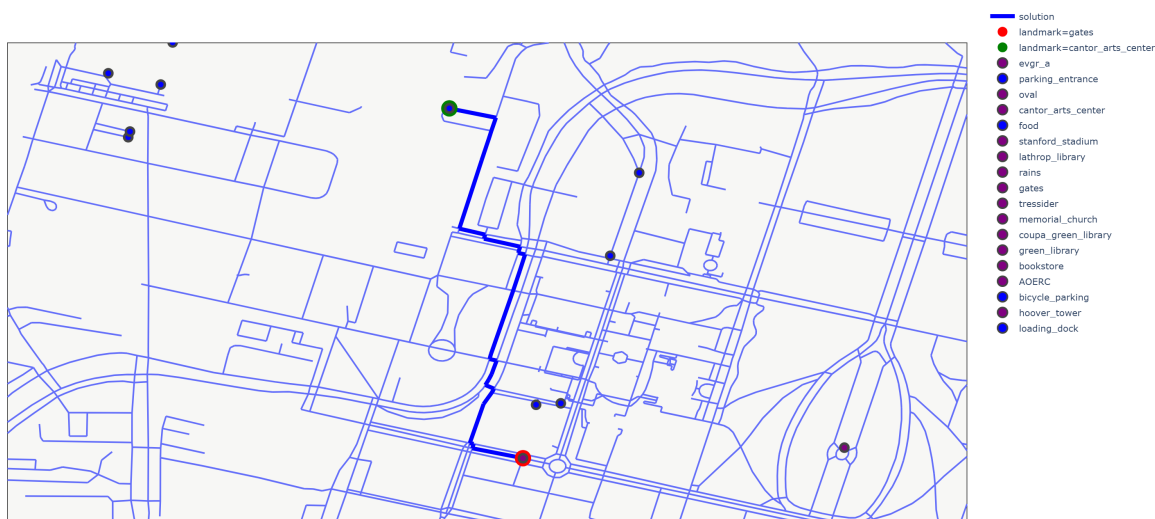
The first setting is the useful one: start at EVGR-A with end tag `landmark=green_library`, which comes out at 606.0 m over 38 locations.

2b (useful): EVGR-A to Green Library



The second is the one where the answer is not desirable: start at Gates with end tag `amenity=food`, 659.2 m, finishing at the Cantor Arts Center.

2b (undesirable): Gates to the nearest amenity=food



What I learned is that the cost is pure distance, so everything that actually slows a walk down is free. My EVGR-A to Green Library route is 606.0 m and passes 2 marked crossings, while the Gates route is almost the same length at 659.2 m but passes 10 crossings and a bus stop, so in practice the second walk takes noticeably longer and the search has no

way to tell the two apart. I also found the locations are spaced very unevenly, from 1.85 m to 98.8 m apart on the same path, so the number of steps in a path says nothing about how far it is.

What is wrong is that asking Gates for the nearest `amenity=food` returns the Cantor Arts Center at 659.2 m, while Tressider, which is the actual food court, sits at 965.9 m and never gets picked. Only four locations on the entire map carry `amenity=food`, and two of them are the art museum and the football stadium, because the tag is attached to the landmark's own OSM node, so one cafe inside a large building ends up labelling the whole building as food. The modelling assumption that breaks is that a tag on a single node stands for the whole place, and that a place I can reach is a place that is open and serving.

2.c

If everyone plans with the same system, the users all get sent down the same shortest path, and the congestion that creates never shows up in the cost, because the cost is only distance. The people who do not use it are hit whenever their own route intersects that optimal path, so quiet side streets that were never built for through traffic end up carrying it, and they get none of the benefit since they never opted in. The mismatch is that the model prices an edge as a fixed number when the real price depends on how many people are on it at the time, and the system has no way of knowing that its own advice is what put them there. One way to reduce that is to put actual speed into the cost rather than distance alone, so a road that is slow or already saturated costs more and the system spreads people out instead of funnelling them all onto one route.

3.b

The maximum number of states UCS could visit is

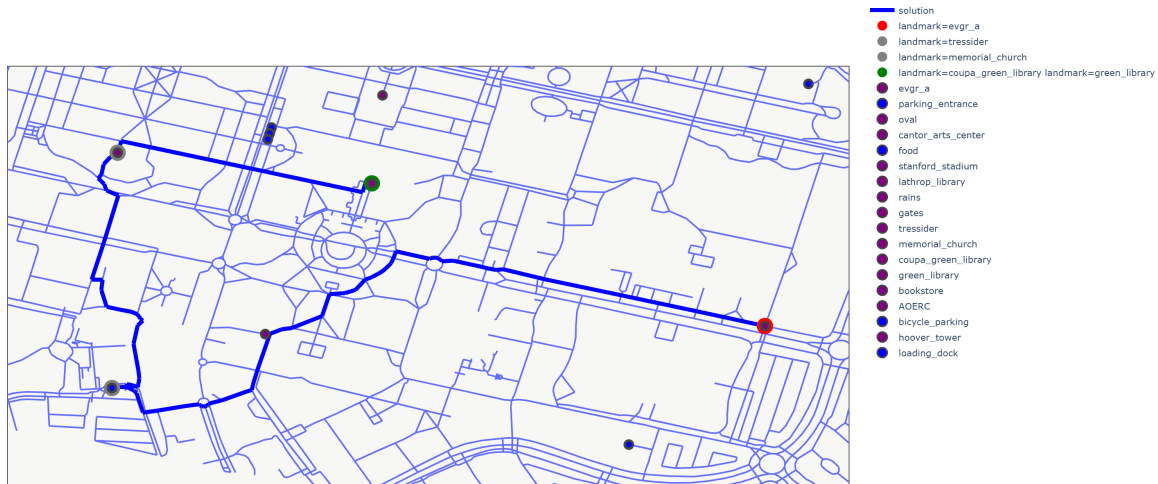
$$n \cdot 2^k.$$

A state is a location together with the set of waypoint tags covered so far, so there are n choices for the location, and for the memory each of the k tags is independently either covered or not, which gives 2^k possible sets. Multiplying the two independent choices gives $n \cdot 2^k$.

3.c

The waypoint tags are `landmark=memorial_church` and `landmark=tressider`, starting at EVGR-A with end tag `landmark=green_library`, and the path comes out at 1815.6 m over 130 locations.

3c: EVGR-A, via Memorial Church and Tressider, to Green Library



The order surprised me. I listed the church first, but the search went to Tressider at 991.5 m, then the church at 1464.8 m, then the library at 1815.6 m, which is the point of leaving the waypoints unordered: I do not have to guess the ordering, and here my guess would have been the more expensive one. What did match my expectation is the cost, since the same trip with no waypoints is 606.0 m, so adding two stops tripled it, which is what you get when the stops sit off the straight line between start and end.

What the path fails to capture is anything other than metres. It has no idea that Tressider closes in the evening, that the church is worth more time than a junction, or that I would rather eat before settling into the library than after, and it is one way, so it never accounts for getting home. A route that is optimal in distance can still be the wrong plan for the day.

3.d

Between a drop-off and the next pick-up I would insert a rest waypoint, tagged for a restroom, food, or somewhere to park for ten minutes, in the same way problem 4c joins a fake node to every end location at zero cost so the search finds whichever one is nearest. Here the cost would be small rather than zero, so a break is cheap enough that the route takes one but not so free that the driver is sent miles out of the way for it. To pick sensible waypoints I would want the driver's age and years of driving experience, since a new driver and a driver in their seventies do not need the same break pattern or the same length of shift. Those same two fields are exactly what a company could turn against them: knowing a driver's age lets dispatch quietly stop sending work to older drivers, which is age discrimination dressed up as routing, and it never has to be written down anywhere because it only shows up as fewer jobs.

5.a

5.b

6.a

My product is Claude Code, carried over from Assignment 1, and it is marketed through non-consumer channels rather than advertising, with new releases announced through Anthropic's own newsroom and social media and then carried outward by news outlets. The main surface is its product page at claude.com/product/claude-code, which leads with "Hand Claude a bug fix, test, or multi-day migration. Steer and review from your terminal, IDE, Slack, or web" and puts the one-line install straight on the page, so discovery and installation happen in the same step. Anthropic also runs its own developer conference, Code with Claude, which in 2026 ran in San Francisco, London and Tokyo with sessions from companies such as Spotify and Delivery Hero on how they restructured their engineering around it ([MIT Technology Review](#)). The other route in is the workplace rather than the individual: Claude Code is sold into businesses on Team and Enterprise plans at roughly \$20 to \$25 per seat plus usage, with admin controls, audit logs and SCIM (claude.com/pricing), so a lot of developers first meet it because their employer signed an agreement and it turned up in their existing workflow. Beyond that, most discovery is word of mouth inside developer communities, where Reddit threads trading multi-agent terminal setups spread the advanced features further than any official page does. Anthropic does advertise at brand level, including two Super Bowl spots in February 2026 attacking OpenAI's decision to put ads in ChatGPT, which lifted the Claude app from 41st to the top ten on the US App Store and daily active users by about 11 percent ([CNBC](#)), but that campaign was for Claude as a whole and never mentions Claude Code.

6.b

6.c

6.d